

APOSTILA DE TAREFAS

SQL Server

26 Desafios Progressivos

Dataset baseado na arquitetura de dados do LinkedIn

usuarios empresas vagas conexoes posts candidaturas habilidades

26 Tarefas | 4 Niveis | Diagrama ER | Criterios de Aceite

Como usar esta apostila

Esta apostila contém 26 tarefas práticas para consolidar os conceitos do Workshop SQL Server. As tarefas utilizam um dataset próprio — a rede social profissional LinkedIn — completamente diferente do dataset de ensino (Uber), forçando você a aplicar os conceitos em um novo contexto sem copiar consultas do material de aula.

Cada tarefa indica o módulo de referência da apostila de ensino, o nível de dificuldade e critérios de aceite objetivos. Resolva as tarefas em ordem: cada grupo prepara o terreno para o seguinte.

Nível	Tarefas	Conceitos cobertos
Básico	1 a 8	DDL, DML, SELECT, filtros, paginação, JOINS simples
Intermediário	9 a 16	JOINS complexos, subqueries, CASE WHEN, índices, SPs, funções
Avançado	17 a 22	Window Functions, ROLLUP, CTEs, Views, MERGE, PIVOT
Projeto Final	23 a 26	Integração de todos os módulos, relatórios, otimização, Docker

Schema do banco linkedin_db — Diagrama Relacional

O diagrama abaixo define a estrutura completa do banco linkedin_db. Use-o como referência para criar o banco, as tabelas, colunas, tipos de dados e todas as constraints de integridade. NÃO há DDL pronto — você deve escrever cada comando CREATE TABLE a partir deste diagrama.

Dica: Comece pelos registros sem FK (setores, habilidades) e avance para as tabelas dependentes. Respeite a ordem de criação para evitar erros de FK. Consulte o Módulo 3 da apostila de ensino para a sintaxe de CREATE TABLE, IDENTITY, UNIQUE, CHECK, FOREIGN KEY e DEFAULT.

Legenda de ícones e tags utilizadas no diagrama

Ícone / Tag	Significado	Sintaxe T-SQL equivalente
PK (laranja)	Chave primária	CONSTRAINT pk_... PRIMARY KEY

FK (azul)	Chave estrangeira — indica a tabela referenciada	CONSTRAINT fk_... FOREIGN KEY (...) REFERENCES ...
NOT NULL	Campo obrigatorio	nome_coluna TIPO NOT NULL
UNIQUE	Valor unico na tabela	CONSTRAINT uq_... UNIQUE (coluna)
AUTO	IDENTITY — autoincremento	INT IDENTITY(1,1)
DEFAULT: X	Valor padrao quando nao informado	nome_coluna TIPO DEFAULT X
CHECK	Restricao de dominio (lista/range)	CONSTRAINT chk_... CHECK (coluna IN (...))
nullable	Campo opcional — aceita NULL	nome_coluna TIPO NULL

Pagina 1 do diagrama — Entidades de Perfil

setores · habilidades · empresas · usuarios · usuario_habilidades · experiencias · conexoes

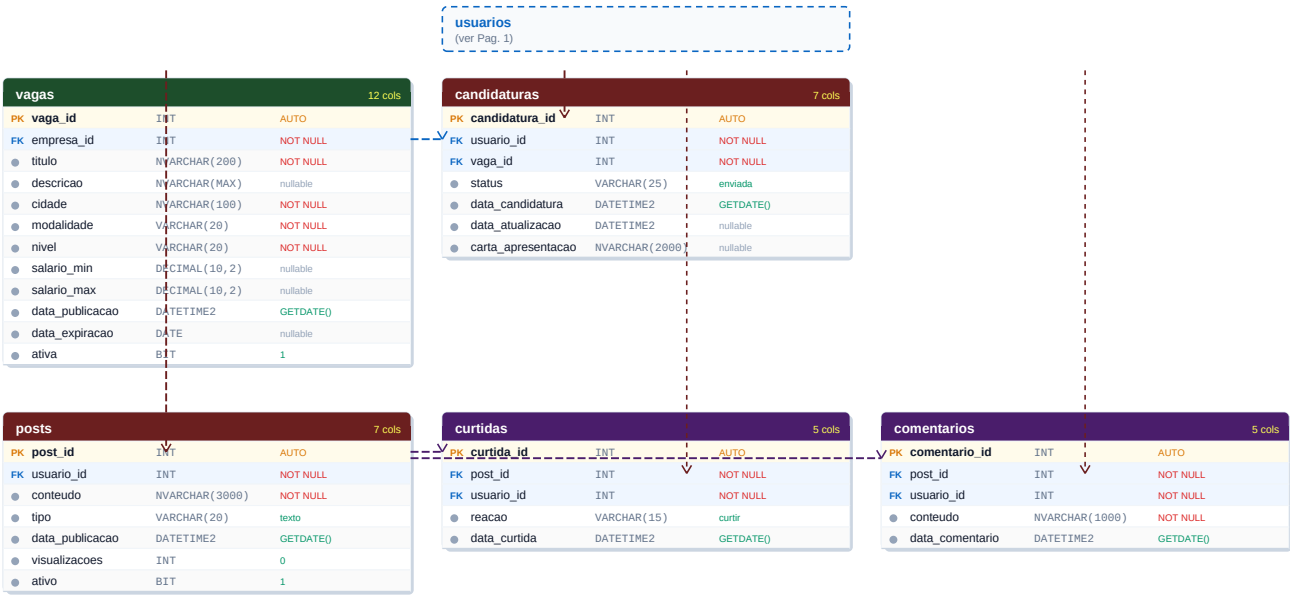


● PK Chave Primaria ● FK Chave Estrangeira ● NOT NULL ● UNIQUE ● DEFAULT ● CHECK

As linhas tracejadas indicam relacionamentos via chave estrangeira. A seta aponta para a tabela que POSSUI a FK (tabela filha).
Conexoes tem duas FK para usuarios (origem e destino) — auto-relacionamento.

Pagina 2 do diagrama — Entidades de Engajamento

usuarios (ref.) · vagas · candidaturas · posts · curtidas · comentarios



O quadrado tracejado 'usuarios (ver Pag. 1)' e apenas uma referencia visual. A tabela usuarios esta definida completamente no diagrama anterior.

Resumo de todos os relacionamentos

Use esta tabela como checklist para verificar se todas as FOREIGN KEYs foram implementadas corretamente.

Tabela filha (possui FK)	Coluna FK	Tabela pai (referenciada)	Coluna PK	Cardinalidade
empresas	setor_id	setores	setor_id	N empresas : 1 setor
usuario_habilidades	usuario_id	usuarios	usuario_id	N : N via tabela pivot
usuario_habilidades	habilidade_id	habilidades	habilidade_id	N : N via tabela pivot
experiencias	usuario_id	usuarios	usuario_id	N experiencias : 1 usuario
experiencias	empresa_id	empresas	empresa_id	N experiencias : 1 empresa
conexoes	usuario_origem	usuarios	usuario_id	N : N reflexivo
conexoes	usuario_destino	usuarios	usuario_id	N : N reflexivo
vagas	empresa_id	empresas	empresa_id	N vagas : 1 empresa
candidaturas	usuario_id	usuarios	usuario_id	N candidaturas : 1 usuario
candidaturas	vaga_id	vagas	vaga_id	N candidaturas : 1 vaga
posts	usuario_id	usuarios	usuario_id	N posts : 1 usuario
curtidas	post_id	posts	post_id	N curtidas : 1 post
curtidas	usuario_id	usuarios	usuario_id	N curtidas : 1 usuario
comentarios	post_id	posts	post_id	N comentarios : 1 post
comentarios	usuario_id	usuarios	usuario_id	N comentarios : 1 usuario

Atencao: Ordem de criacao recomendada: (1) setores, (2) habilidades, (3) empresas, (4) usuarios, (5) usuario_habilidades, (6) experiencias, (7) conexoes, (8) vagas, (9) candidaturas, (10) posts, (11) curtidas, (12) comentarios.

SECAO 1 — BASICO

Tarefas 01 a 08 | Modulos 1 a 4

01

Criacao do Schema a partir do Diagrama

Modulo de referencia: Modulo 3 — DDL

Basico

Contexto: Usando apenas o diagrama relacional desta apostila (sem DDL pronto), voce deve criar o banco e todas as tabelas com suas constraints.

1. Crie o banco linkedin_db com collation Latin1_General_CI_AI.
2. Escreva e execute os 12 comandos CREATE TABLE na ordem correta (respeite as dependencias de FK).
3. Implemente TODAS as constraints visíveis no diagrama: PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK, DEFAULT e NOT NULL.
4. Adicione a coluna linkedin_url VARCHAR(200) UNIQUE na tabela usuarios via ALTER TABLE.
5. Crie a tabela mensagens (mensagem_id PK IDENTITY, remetente_id FK, destinatario_id FK, conteudo NVARCHAR(1000), data_envio DATETIME2, lida BIT DEFAULT 0) — nao esta no diagrama, projete voce mesmo.
6. Verifique com SELECT * FROM INFORMATION_SCHEMA.TABLES e INFORMATION_SCHEMA.COLUMNS.

Critérios de aceite:

- + Todas as 13 tabelas criadas sem erros de FK ou constraint
- + Tentativa de inserir email duplicado em usuarios gera erro de UNIQUE
- + Tentativa de inserir status invalido em conexoes gera erro de CHECK

02**Insercao do Dataset Base**

Modulo de referencia: Modulo 3 — DDL / Modulo 4 — DML

Basico

Contexto: Com o schema criado a partir do diagrama, popule o banco com dados realistas para as proximas tarefas.

1. Insira 8 setores: Tecnologia, Financas, Saude, Educacao, Varejo, Logistica, Marketing, Consultoria.
2. Insira pelo menos 12 empresas distribuidas entre os setores, com tamanhos variados.
3. Insira pelo menos 30 usuarios com titulo_perfil, cidade e data_cadastro variados.
4. Insira pelo menos 15 habilidades (tecnicas e soft skills) e associe-as a usuarios com niveis variados — cada usuario deve ter entre 3 e 7 habilidades.
5. Insira pelo menos 25 vagas distribuidas entre empresas, modalidades e niveis.
6. Insira pelo menos 50 conexoes entre usuarios com status variados.
7. Insira pelo menos 40 posts, 30 candidaturas e 80 curtidas.

Critérios de aceite:

- + Insercao sem violacoes de constraint (teste inserir candidatura duplicada — deve gerar erro UNIQUE)
- + Pelo menos 1 usuario com mais de 5 conexoes aceitas
- + Pelo menos 1 empresa com mais de 3 vagas ativas

03**Consultas Basicas de Selecao**

Modulo de referencia: Modulo 4 — DML

Basico

Contexto: O time de produto precisa de consultas simples para validar os dados e responder perguntas de negocio iniciais.

1. Liste todos os usuarios premium cadastrados nos ultimos 6 meses, ordenados por nome.
2. Exiba as 10 vagas com maior salario_max: titulo, empresa_id, cidade, modalidade, salario_max.
3. Liste todas as conexoes com status 'pendente' ha mais de 7 dias (use DATEDIFF).
4. Mostre posts com mais de 100 visualizacoes, ordenados por visualizacoes decrescente.
5. Liste usuarios sem nenhuma habilidade cadastrada (use NOT EXISTS).
6. Pagine os posts: 10 por pagina, ordenados por data_publicacao decrescente. Exiba a pagina 3 com OFFSET FETCH.

Critérios de aceite:

- + Filtro de 6 meses usa DATEADD(MONTH, -6, GETDATE()) corretamente
- + Paginacao retorna exatamente 10 registros (ou menos se nao houver suficientes na pagina 3)
- + NOT EXISTS e mais seguro que NOT IN com NULLs — documente a diferenca

04

Operadores, Filtros e CASE WHEN

Modulo de referencia: Modulo 4 — DML

Basico

Contexto: O time de dados precisa de relatorios classificados para entender o perfil das vagas e usuarios da plataforma.

1. Liste vagas com salario entre R\$ 5.000 e R\$ 15.000, modalidade remoto, nivel Pleno ou Senior.
2. Classifique cada vaga por faixa salarial com CASE WHEN: 'Ate 5k', '5k a 10k', '10k a 20k', 'Acima de 20k'. Use ISNULL para vagas sem salario informado.
3. Classifique usuarios por senioridade: 'Iniciante' (0-1 experiencias), 'Em crescimento' (2-3), 'Experiente' (4+) — use subquery dentro do CASE.
4. Liste empresas cujo nome contenha 'Tech' ou 'Digital' (LIKE) e estejam no setor de Tecnologia.

Critérios de aceite:

- + CASE WHEN cobre todos os casos sem sobreposicao, incluindo NULLs
- + Filtro de vagas combina BETWEEN, IN e AND corretamente
- + Classificacao de usuarios nao duplica registros

05

UPDATE e DELETE com Seguranca

Modulo de referencia: Modulo 4 — DML

Basico

Contexto: O time de operacoes precisa realizar manutencao de dados: desativar conteudo expirado, atualizar registros e remover dados obsoletos.

1. Expire vagas com data_expiracao anterior a hoje: atualize ativa = 0. Antes, execute um SELECT com os mesmos criterios.
2. Atualize candidaturas para 'reprovada' onde a vaga esteja inativa e o status ainda seja 'enviada' ou 'em_revisao' (use UPDATE com subquery ou JOIN).
3. Remova curtidas de posts inativos (ativo = 0) usando DELETE com subquery.
4. Atualize visualizacoes somando 1 para todos os posts publicados ha mais de 30 dias e com pelo menos 1 curtida.

Critérios de aceite:

- + Todo UPDATE e DELETE foi precedido por SELECT com os mesmos criterios
- + Nenhuma FK violada nas operacoes de DELETE
- + UPDATE de candidaturas usa subquery ou JOIN corretamente

06

JOINS Fundamentais

Modulo de referencia: Modulo 5 — Consultas Avancadas

Basico

Contexto: O dashboard de vagas precisa de consultas que cruzem dados de multiplas tabelas.

1. Liste todas as vagas ativas com: titulo, nome da empresa, setor, cidade, modalidade, nivel e faixa salarial formatada como 'R\$ X.XXX - R\$ Y.YYY'.
2. Mostre todos os usuarios com suas candidaturas: nome, titulo da vaga, nome da empresa, status e data.
3. Encontre empresas sem nenhuma vaga ativa (LEFT JOIN + IS NULL).
4. Liste usuarios e a quantidade de conexoes aceitas (inclua usuarios com 0 conexoes).

Critérios de aceite:

- + Faixa salarial formatada com CONCAT + CAST/FORMAT
- + Usuarios com 0 conexoes aparecem com COUNT = 0
- + LEFT JOIN + IS NULL retorna corretamente empresas sem vagas

07

Subqueries e EXISTS

Modulo de referencia: Modulo 5 — Consultas Avancadas

Basico

Contexto: O algoritmo de recomendacao precisa identificar usuarios qualificados com base em habilidades e historico.

1. Liste vagas para as quais o usuario_id = 5 ainda nao se candidatou (NOT EXISTS).
2. Liste usuarios com habilidade 'SQL' em nivel 'Avancado' ou 'Especialista' (subquery com IN).
3. Encontre posts com curtidas de pelo menos 3 usuarios diferentes nos ultimos 7 dias (subquery com HAVING no WHERE).
4. Liste empresas com mais vagas abertas que a media geral de vagas por empresa.

Critérios de aceite:

- + NOT EXISTS e NOT IN produzem mesmo resultado — documente a diferenca com NULLs
- + Subquery com HAVING usada corretamente no WHERE
- + Media usa subquery escalar

Basico

Contexto: O time de analytics precisa de metricas basicas sobre a plataforma para o relatorio semanal.

1. Calcule a idade media dos perfis em dias (DATEDIFF entre data_cadastro e hoje).
2. Identifique o mes/ano com mais cadastros de usuarios (GROUP BY YEAR() e MONTH()).
3. Liste as 5 habilidades mais associadas a usuarios com contagem e percentual do total.
4. Mostre a distribuicao de candidaturas por status: quantidade e % do total.
5. Liste usuarios cujo titulo_perfil contenha 'Engenheiro', 'Analista' ou 'Desenvolvedor' (LIKE com OR).

Critérios de aceite:

- + Percentual usa CAST(...AS FLOAT) para evitar divisao inteira
- + GROUP BY em funcoes de data funciona corretamente
- + Top 5 habilidades usa TOP 5 com ORDER BY COUNT(*) DESC

SECAO 2 — INTERMEDIARIO

Tarefas 09 a 16 | Modulos 5 a 7

09

JOINS Avancados e Multi-tabela

Modulo de referencia: Modulo 5 — Consultas Avancadas

Intermediario

Contexto: O time de produto precisa de relatorios que cruzem profundamente perfis, vagas e interacoes.

1. Crie um relatorio completo de candidaturas: nome, email, titulo do perfil, titulo da vaga, empresa, setor, cidade da vaga, modalidade, nivel, status e data.
2. Liste usuarios e suas conexoes de 2 grau (amigos de amigos ainda nao conectados) — use SELF JOIN em conexoes.
3. Encontre usuarios que se candidataram a vagas em empresas onde ja trabalharam (cruzar candidaturas com experiencias).
4. Para cada empresa: nome, setor, total de vagas, total de candidaturas, taxa de conversao (aprovadas / total). Use NULLIF para evitar divisao por zero.

Critérios de aceite:

- + Relatorio completo sem linhas duplicadas por multiplicacao de JOINS
- + Conexoes de 2 grau excluem conexoes ja existentes (NOT EXISTS)
- + Taxa de conversao trata divisao por zero

10

Índices — Criação e Análise de Performance

Módulo de referência: Módulo 6 — Índices

Intermediário

Contexto: Consultas de busca de vagas e feed de posts estão lentas. Crie os índices corretos e meça o impacto.

1. Crie um índice non-clustered para busca de vagas por empresa e data: IX_vagas_empresa_data em vagas(empresa_id, data_publicacao DESC) INCLUDE (titulo, modalidade, nivel).
2. Crie um índice filtrado para vagas ativas: IX_vagas_ativas em vagas(empresa_id, data_publicacao) WHERE ativa = 1.
3. Crie índice para candidaturas por usuário: IX_cand_usuario em candidaturas(usuario_id, status) INCLUDE (vaga_id, data_candidatura).
4. Execute com SET STATISTICS IO ON a mesma consulta antes e depois de cada índice. Registre os logical reads em tabela comparativa.
5. Use o Execution Plan para confirmar Index Seek (não Table Scan) após criar os índices.

Critérios de aceite:

- + Tabela comparativa: tabela, índice, query, logical reads antes, logical reads depois
- + Índice filtrado criado com WHERE ativa = 1
- + Execution Plan confirma Index Seek após criação dos índices

11

Stored Procedure — Busca de Vagas

Módulo de referência: Módulo 7 — SPs e Funções

Intermediário

Contexto: O motor de busca precisa de uma SP com múltiplos filtros opcionais.

1. Crie sp_buscar_vagas com parâmetros opcionais (NULL = ignorar): @palavra_chave NVARCHAR, @cidade NVARCHAR, @modalidade VARCHAR, @nivel VARCHAR, @salario_min DECIMAL, @empresa_id INT.
2. Retorne: titulo, empresa, setor, cidade, modalidade, nivel, salario_min, salario_max, data_publicacao — somente vagas ativas.
3. Implemente filtros opcionais: o filtro só é aplicado quando o parâmetro não for NULL.
4. Adicione TRY/CATCH com THROW para parâmetros inválidos (ex: salario_min negativo).
5. Teste com: todos nulos, apenas cidade = 'São Paulo', apenas nivel = 'Senior' e combinação de 3 filtros.

Critérios de aceite:

- + SP testada 5 vezes com diferentes combinações — todas corretas
- + Filtros nulos não afetam o resultado
- + THROW para salario_min negativo funciona corretamente

12

Stored Procedure — Candidatura com Validacao

Modulo de referencia: Modulo 7 — SPs e Funcoes

Intermediario

Contexto: O fluxo de candidatura precisa de validacoes de negocio antes de registrar.

1. Crie `sp_registrar_candidatura(@usuario_id INT, @vaga_id INT, @carta NVARCHAR)`.
2. Valide: (1) usuario existe e ativo, (2) vaga existe e ativa, (3) candidatura nao duplicada, (4) vaga nao expirada.
3. Use `THROW` com codigos 50010 a 50013 para cada validacao.
4. Insira dentro de `BEGIN TRANSACTION` com `TRY/CATCH` e retorne `candidatura_id` como `OUTPUT`.

Critérios de aceite:

- + 4 cenarios de erro testados individualmente
- + `ROLLBACK` ocorre corretamente em caso de erro
- + `candidatura_id` retornado no `OUTPUT`

13

Funcoes Customizadas

Modulo de referencia: Modulo 7 — SPs e Funcoes

Intermediario

Contexto: O time precisa de funcoes reutilizaveis para calcular metricas e filtrar dados de forma padronizada.

1. Crie `fn_score_perfil(@usuario_id INT)` retornando score: +10 por habilidade, +15 por experiencia, +5 por conexao aceita, +20 se premium.
2. Crie TVF `fn_conexoes_usuario(@usuario_id INT)` retornando dados completos das conexoes aceitas.
3. Crie tabela `vaga_habilidades` (`vaga_id` FK, `habilidade_id` FK), popule com dados e crie TVF `fn_vagas_recomendadas(@usuario_id INT)` retornando vagas com pelo menos 2 habilidades em comum.
4. Use `fn_score_perfil` em `SELECT` para listar os top 10 usuarios por score.

Crterios de aceite:

- + `fn_score_perfil` retorna scores diferentes para perfis diferentes
- + TVFs usaveis em `FROM` como tabelas
- + `fn_vagas_recomendadas` nao recomenda vagas ja candidatas

14

Aggregations — Metricas da Plataforma

Modulo de referencia: Modulo 8 — Aggregations

Intermediario

Contexto: O time de growth precisa de um painel de metricas consolidadas para apresentar ao board.

1. Para cada empresa: total de vagas por modalidade (presencial/hibrido/remoto) com GROUP BY.
2. Calcule o NPS de vagas por setor: % candidaturas aprovadas menos % reprovadas, por setor.
3. Identifique a hora do dia com mais publicacoes de vagas (DATEPART(HOUR, ...)).
4. Use ROLLUP para calcular subtotais de candidaturas por empresa e total geral.

Critérios de aceite:

- + ROLLUP gera subtotal por empresa e linha de total geral
- + NPS usa CAST para DECIMAL antes da divisao
- + DATEPART(HOUR) aplicado corretamente

15

Exportacao e Backup

Modulo de referencia: Modulo 10 — Exportacao e Backup

Intermediario

Contexto: O time de infraestrutura precisa implementar rotinas de exportacao e backup.

1. Use BCP para exportar a tabela vagas para vagas_export.csv (titulo, empresa_id, cidade, modalidade, nivel, salario_min, salario_max).
2. Execute BACKUP FULL com compressao e verifique com RESTORE VERIFYONLY.
3. Documente estrategia de backup: Full/Diferencial/Log, frequencia, RPO e RTO.
4. Restaure em linkedin_db_test com RESTORE DATABASE usando MOVE para remapear os arquivos.

Critérios de aceite:

- + CSV exportado com separador virgula e cabeçalho
- + RESTORE VERIFYONLY conclui sem erros
- + Estrategia de backup com RPO e RTO definidos

Intermediario

Contexto: O time de devops precisa que o ambiente seja 100% reproduzível via Docker.

1. Crie docker-compose.yml com SQL Server 2022 e Adminer, usando .env para credenciais.
2. Crie pasta ./scripts/ com: 01_create_schema.sql, 02_seed_data.sql, 03_indexes.sql, 04_procedures.sql.
3. Monte ./scripts como volume e execute os 4 scripts via docker exec na ordem correta.
4. Documente strings de conexão para Python (pyodbc), Node.js (mssql) e .NET no README.md.

Critérios de aceite:

- + docker-compose up -d sobe os dois containers sem erro
- + Scripts executados em ordem sem erros de FK
- + README.md executável do zero

SECAO 3 — AVANÇADO

Tarefas 17 a 22 | Modulos 8 e 9

17

Window Functions — Feed e Engajamento

Modulo de referencia: Modulo 8 — Window Functions

Avancado

Contexto: O algoritmo de feed usa ranking e comparacoes temporais para decidir quais posts exibir.

1. Use ROW_NUMBER() PARTITION BY usuario_id para retornar apenas o post mais recente de cada usuario.
2. Use DENSE_RANK() para ranquear usuarios por curtidas recebidas nos ultimos 30 dias. Exiba top 10.
3. Use LAG() para calcular o intervalo em dias entre posts consecutivos de cada usuario. Identifique usuarios com intervalo medio acima de 30 dias.
4. Calcule o percentual de participacao de cada setor no total de vagas usando SUM() OVER ().
5. Calcule o running total de candidaturas por data, particionado por empresa.

Critérios de aceite:

- + ROW_NUMBER() com WHERE rn = 1 retorna exatamente 1 post por usuario
- + LAG retorna NULL no primeiro post — trate com ISNULL
- + Percentual soma 100% para todos os setores

18

CTEs — Analise de Rede de Conexoes

Modulo de referencia: Modulo 9 — CTEs

Avancado

Contexto: O time de data science precisa entender a estrutura da rede para calcular alcance e influencia.

1. Escreva CTE recursiva que expanda a rede de um usuario ate o 3 grau. Use OPTION (MAXRECURSION 4).
2. Use CTEs multiplas para: total de conexoes diretas, total de conexoes de 2 grau unicas e quantos sao premium.
3. Identifique os 5 usuarios mais influentes: (conexoes_aceitas * 2) + (posts_proprios * 3) + (curtidas_recebidas * 1).

Critérios de aceite:

- + CTE recursiva tem anchor member e recursive member definidos
- + MAXRECURSION aplicado corretamente
- + Score de influencia sem duplicacoes

Avancado

Contexto: Para simplificar o acesso dos times de produto e analytics, crie views encapsulando as queries complexas.

1. Crie vw_perfil_completo: nome, email, titulo, cidade, qtd conexoes aceitas, qtd posts, total curtidas recebidas, fn_score_perfil, habilidades como STRING_AGG.
2. Crie vw_vagas_completas: titulo, empresa, setor, cidade, modalidade, nivel, faixa salarial, dias desde publicacao, total candidaturas.
3. Crie vw_empresa_dashboard: nome, setor, tamanho, vagas ativas, total candidaturas, taxa de aprovacao (com NULLIF), media de dias para preenchimento.
4. Teste cada view com SELECT + WHERE + ORDER BY.

Critérios de aceite:

- + STRING_AGG correto e sem duplicatas
- + Cada view retorna 1 linha por entidade principal
- + NULLIF trata divisao por zero em vw_empresa_dashboard

Avancado

Contexto: O time precisa sincronizar dados externos e gerar relatorios em formato matricial.

1. Crie tabela habilidades_mercado (habilidade_id FK, nome, demanda_vagas INT). Use MERGE para sincronizar com dados reais de vaga_habilidades.
2. Crie PIVOT com vagas por empresa (linhas) x modalidade (colunas). Substitua NULLs por 0 com ISNULL.
3. Crie segundo PIVOT: candidaturas por setor (linhas) x status (colunas).

Critérios de aceite:

- + MERGE atualiza e insere sem duplicatas em multiplas execucoes
- + PIVOT exibe 0 para combinacoes sem dados
- + Linha de total clara nos PIVOTs

21

Otimizacao de Queries Problematicas

Modulo de referencia: Modulo 6 — Indices e Performance

Avancado

Contexto: Voce recebeu 3 queries legadas que causam lentidao em producao. Analise e otimize cada uma.

1. Query A: `SELECT * FROM vagas WHERE YEAR(data_publicacao) = 2024`. Reescreva usando range filter (sem funcao na coluna indexada).
2. Query B: `SELECT * FROM posts ORDER BY visualizacoes DESC` (sem filtro, sem projecao). Reescreva com projecao adequada, filtro e paginacao.
3. Query C: `SELECT u.nome, (SELECT COUNT(*) FROM conexoes WHERE usuario_origem = u.usuario_id) FROM usuarios u`. Reescreva com JOIN e GROUP BY.
4. Para cada: versao original, versao otimizada, logical reads antes/depois, Execution Plan.

Critérios de aceite:

- + Query A reescrita com BETWEEN ou $\geq$ / $\leq$ sem YEAR() na coluna
- + Query C elimina subquery correlacionada
- + Reducao de logical reads documentada em pelo menos 2 queries

22

Analise de Cohort de Retencao

Modulo de referencia: Modulos 8 e 9

Avancado

Contexto: O time de growth quer entender retencao: quantos usuarios cadastrados em cada mes continuam ativos nos meses seguintes.

1. Defina o mes de cadastro de cada usuario como cohort de aquisicao.
2. Para cada cohort, calcule usuarios que fizeram ao menos 1 acao (post ou candidatura) em M+1, M+2, ..., M+6.
3. Calcule o percentual de retencao: $\text{usuarios ativos no mes} / \text{total do cohort} * 100$.
4. Apresente como tabela: cohort (mes/ano) x mes_de_atividade (M+1 a M+6) x % retencao.
5. Solucao deve usar apenas SQL set-based — proibido cursores ou loops.

Critérios de aceite:

- + Cohort correto para cada usuario
- + Percentual sem divisao por zero
- + Apenas CTEs e/ou Window Functions — sem cursores

SECAO 4 — PROJETO FINAL

Tarefas 23 a 26 | Integracao de todos os modulos

As tarefas a seguir integram todos os conceitos do workshop e simulam entregas reais de um time de engenharia de dados em uma plataforma como o LinkedIn.

23

Dashboard Executivo — SQL Puro

Modulo de referencia: Todos os modulos

Avancado

Contexto: O CEO precisa de um dashboard com KPIs gerado 100% em SQL, sem ferramentas de BI.

1. Use CTEs multiplas e UNION ALL para retornar em 1 result set (formato: metrica | valor | variacao_pct):
2. Total de usuarios ativos hoje vs semana passada (variacao %).
3. Total de vagas abertas hoje vs semana passada.
4. Total de candidaturas nos ultimos 7 dias vs 7 dias anteriores.
5. Setor com mais vagas abertas, usuario com mais posts no mes, taxa de conversao global do mes.
6. Apenas SQL set-based — sem cursores. NULLIF em todas as divisoes.

Critérios de aceite:

- + Exatamente 1 linha por metrica no result set
- + Variacoes percentuais com NULLIF para divisao por zero
- + Nenhum cursor ou loop utilizado

24

Sistema de Recomendacao de Vagas por SP

Modulo de referencia: Modulos 7, 8 e 9

Avancado

Contexto: O time de produto quer uma SP que recomende vagas personalizadas baseadas no perfil do usuario.

1. Crie sp_recomendar_vagas @usuario_id INT, @top_n INT = 10.
2. Score: +30 por habilidade em comum, +20 se cidade da vaga = cidade do usuario, +15 se nivel compativel, +10 se conexao do usuario trabalha na empresa.
3. Excluir vagas ja candidatas ou inativas.
4. Retornar: titulo, empresa, cidade, modalidade, nivel, score_relevancia DESC.
5. TRY/CATCH e validacao de @top_n positivo.

Critérios de aceite:

- + SP testada para usuarios com perfis diferentes — scores distintos
- + Vagas ja candidatas NAO aparecem
- + Usuario com mais habilidades em comum recebe score maior

25

Relatorio Consolidado de Contratacoes

Modulo de referencia: Modulos 8, 9 e 10

Avancado

Contexto: O time de RH precisa de relatorio completo de vagas, candidatos e taxas de contratacao.

1. Crie sp_relatorio_rh @empresa_id INT, @data_inicio DATE, @data_fim DATE.
2. Por vaga: titulo, total candidaturas, distribuicao por status (PIVOT), tempo medio ate primeira candidatura e tempo medio ate aprovacao.
3. Por empresa: total candidaturas, aprovacoes, taxa de aprovacao, vaga mais disputada, habilidades dos aprovados via STRING_AGG.
4. Use pelo menos 3 CTEs nomeadas para organizar os calculos.
5. Teste com empresa especifica e com @empresa_id = NULL (todas as empresas).

Critérios de aceite:

- + SP funciona com empresa_id especifico e NULL
- + PIVOT de status exhibe todos os 6 status
- + STRING_AGG sem duplicatas

Avancado

Contexto: Consolide todo o trabalho em um repositorio organizado, documentado e reproduzível do zero.

1. Organize scripts em: /ddl, /dml_seed, /indexes, /procedures, /functions, /views, /tasks.
2. Crie README.md com descricao, pre-requisitos, setup do zero (docker + scripts) e exemplos.
3. Crie DOCS.md documentando cada SP e funcao: descricao, parametros, exemplo de chamada, resultado esperado.
4. Execute a sequencia completa a partir de um banco zerado usando apenas os scripts do repositorio.
5. Bonus: script Python com pyodbc que executa sp_recomendar_vagas para os 3 primeiros usuarios e salva em CSV.

Critérios de aceite:

- + Repositorio com estrutura de pastas conforme especificado
- + Execucao do zero concluida sem erros manuais
- + README.md executavel: seguir as instrucoes gera ambiente funcional